

Reviewer Pack — Minimal Rank of Algebraic Combinatorial Designs and Communic...

Entry 703776d433f4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 21:48:32 UTC

Minimal Rank of Algebraic Combinatorial Designs and Communication Complexity Rank

Entry ID: 703776d433f4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 21:48:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Design Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex graph G , the minimal rank of an associated algebraic combinatorial design $D(G)$ is lower bounded by the communication complexity rank $\kappa(G)$, such that $r(D(G)) \geq \kappa(G) + \log(n)$.

Rationale (proposer's reasoning):

Algebraic combinatorial designs provide a structured framework for encoding information, potentially capturing non-trivial structure in communication tasks. If the minimal rank of such designs is closely related to communication complexity rank, it suggests that certain aspects of communication can be efficiently represented using these designs.

Taxonomy category: Design_Theoretic_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8c21538557767d95

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each graph G with $n \leq 40$ vertices, the minimal rank of algebraic combinatorial design $D(G)$, $r(D(G))$, is lower bounded by the communication complexity rank $\kappa(G)$ plus $\log(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic combinatorial design" AND "communication complexity rank"
- "minimal rank" INALPHA "algebraic combinatorial design" AND $\kappa(G) \geq \kappa(G) + \log(n)$ - "graph G " AND $r(D(G)) \geq \kappa(G) + \log(n)$

Top relevant hits considered: - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2208.01392v2] The Analytic Minimal Rank Sard Conjecture - [http://arxiv.org/abs/2406.15089v2] Sentences over Random Groups II: Sentences of Minimal Rank - [http://arxiv.org/abs/0905.1394v1] On Longest Cycle C of a graph G via Structures of $G - C$ - [http://arxiv.org/abs/2305.04326v2] Moderate deviations of triangle counts in sparse Erdős-Rényi random graphs $G(n, m)$ and $G(n, p)$ - [http://arxiv.org/abs/1210.7919v3] Tree t-spanners in Outerplanar Graphs via Supply Demand Partition

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        G = [[0] * n for _ in range(n)]
        degree = 2
        if n % degree != 0:
            return None, None
        for i in range(n // degree):
            for j in range(degree):
                u = i * degree + j
                v = (i + 1) % (n // degree) * degree + (j + 1) % degree
                G[u][v] = 1
                G[v][u] = 1
        return G, degree

    def communication_complexity_rank(G):
        n = len(G)
        degree = sum(sum(row) for row in G) // n
        if degree == 0:
            return 0
        rank = 0
        visited = [False] * n
```

```

stack = []
for i in range(n):
    if not visited[i]:
        stack.append(i)
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                rank += 1
                for v in range(n):
                    if G[u][v] == 1 and not visited[v]:
                        stack.append(v)

return rank

def minimal_rank(G):
    n = len(G)
    A = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j] == 1:
                A[i][j] = 1
                A[j][i] = 1

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    rank = 0
    for i in range(n):
        j = -1
        for k in range(m):
            if A[k][i]:
                j = k
                break
        if j == -1:
            continue
        for k in range(i, n):
            A[j][k], A[i][k] = A[i][k], A[j][k]
        rank += 1
        for k in range(m):
            if k != j and A[k][i]:
                for l in range(n):
                    A[k][l] -= A[j][l] * Fraction(A[k][i], A[j][i])

    return rank

return gaussian_elimination(A)

n = random.choice([5, 10, 15, 20, 30, 40])
G, degree = generate_graph(n)
if G is None:
    return {
        "metric_name": "minimal_rank",
        "metric_value": 0,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "degree_not_multiple_of_n"
    }

```

```

kappa_G = communication_complexity_rank(G)
r_DG = minimal_rank(G)

if r_DG >= kappa_G + math.log(n, 2):
    return {
        "metric_name": "minimal_rank",
        "metric_value": r_DG,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "minimal_rank",
        "metric_value": r_DG,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": f"r(D(G))={r_DG}, kappa_G+log(n)={kappa_G + math.log(n, 2)}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std = (sum((x - mean) ** 2 for x in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r >= kappa_G + math.log(n, 2)) / len(results)

    if all(r >= kappa_G + math.log(n, 2) for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction:.2f}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if r < kappa_G + math.log(n, 2))
        print(f"RESULT: FALSIFIED counterexample=\"r(D(G))<{kappa_G + math.log(n, 2)}\" first_fail")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_491fc1da.py", line 126, in <module>
 trial_result = run_trial(seed)

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_491fc1da.py", line 99, in run_trial
    r_DG = minimal_rank(G)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_491fc1da.py", line 84, in minimal_rank
    return gaussian_elimination(A)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_491fc1da.py", line 81, in gaussian_elim
    A[k][l] -= A[j][l] * Fraction(A[k][i], A[j][i])
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13585
2	preregistration	ollama_remote	glm4:latest	0	0	9318
3	novelty	ollama_remote	glm4:latest	0	0	15880
4	novelty	ollama_remote	glm4:latest	0	0	14164
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14158
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8514
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10261
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13591
9	judge	ollama_remote	glm4:latest	0	0	12769

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112240 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/703776d433f4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/703776d433f4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/703776d433f4.tar.gz` (if generated)